

A PROJECT REPORT
on
DEVELOPMENT OF SMART CODE INSPECTION PLATFORM
WITH VULNERABILITY DETECTION SYSTEM

Submitted to
INFOSYS SPRINGBOARD
In partial fulfilment of the requirements of
VIRTUAL INTERNSHIP 7.0 — BATCH 1

By
KANNEBOINA RAJASHEKAR rajashekarkanneboina25@gmail.com

Under the Guidance of
Mr.Keshav Yadav, Infosys Springboard



INFOSYS SPRINGBOARD
VIRTUAL INTERNSHIP PROGRAM 7.0
August 2026

ACKNOWLEDGEMENTS

I would like to express our sincere gratitude to Infosys Springboard and its Virtual Internship 7.0 program team for providing us the opportunity, platform, and structured guidance to design and build this project. Their well-organised milestone framework and continuous evaluation criteria helped us translate the problem statement into a working, end-to-end system.

We are especially thankful to our Internship Coordinator and mentors for their valuable suggestions during every milestone review, for helping us refine the multi-agent architecture, and for their patience in clarifying doubts on secure coding practices, RAG pipelines, and orchestration design.

I also thank our respective colleges and faculty coordinators for supporting our participation in this virtual internship alongside our academic commitments, and our teammates for their collaboration, consistency, and shared ownership of every module of this project.

Finally, I thank my familie and friends for their constant encouragement throughout the duration of this internship.

ABSTRACT

Software development teams routinely struggle with inconsistent code quality, undetected security vulnerabilities, and manual code reviews that are slow, subjective, and unable to scale with growing codebases. Developers rarely have immediate access to expert guidance on secure coding practices during active development, which means quality issues and OWASP-class vulnerabilities are often discovered only late in the development lifecycle.

To address this gap, this project presents the Smart Code Inspection Platform with Vulnerability Detection System (internally referred to during development as the AI Code Review & Security Analysis Agent) — a multi-agent platform that automatically analyses Python and Java source code for code smells, design anti-patterns, and OWASP-standard security vulnerabilities such as SQL Injection, Command Injection, Path Traversal, Insecure Deserialization, Hardcoded Secrets, and Weak Cryptography.

The platform is built around five cooperating agents orchestrated with LangGraph: a Code Analysis Agent and a Security Vulnerability Agent that run in parallel over established static-analysis tools (Pylint, Radon, Bandit for Python; PMD, Checkstyle, SpotBugs for Java) supplemented by a Groq-hosted large language model; a Remediation Agent that produces corrected code examples and fix guidance; a PR Summary Agent that compiles an executive, pull-request-style review; and a Conversational Code Assistant that answers follow-up questions grounded in a Retrieval-Augmented Generation (RAG) pipeline built over an indexed secure-coding knowledge base, including the OWASP Top 10.

A key design decision is that every finding — regardless of which underlying tool produced it — is normalised through a single, configuration-driven severity engine, so that severity labels, OWASP category tags, and score penalties are consistent across the dashboard, chat assistant, database, and exported PDF report. Findings are displayed in a Streamlit-based developer portal with severity counts, a composite health score, a five-axis radar chart (Security, Quality, Maintainability, Reliability, Complexity), and OWASP coverage, and can be exported as a structured PDF report generated with ReportLab.

The system was validated end-to-end across multiple distinct Python and Java code samples covering the OWASP vulnerability classes listed above, exceeding the minimum three-sample demonstration requirement. Overall, the platform demonstrates how static analysis tooling, multi-agent orchestration, and retrieval-grounded large language models can be combined into an explainable, actionable, and scalable alternative to manual code review.

TABLE OF CONTENTS

Certificate	ii
Acknowledgements.....	iii
Abstract.....	iv
List of Figures.....	v
List of Abbreviations	vi
List of Tables	vii
Chapter 1: Introduction	1
1.1 Background.....	1
1.2 Challenges in Manual Code Review	2
1.3 AI-Based Code Review Assistance.....	3
1.4 Motivation for the Proposed Work	3
1.5 Contributions.....	4
1.6 Organization of the Report	4
Chapter 2: Literature Survey	5
2.1 Existing Systems.....	5
2.2 Limitations of Existing Systems.....	6
2.3 Proposed System	6
2.4 Summary.....	7
Chapter 3: System Requirement Specification.....	8
3.1 Hardware Requirements	8
3.2 Software Requirements	8
3.3 Supporting Modules / Libraries.....	9
Chapter 4: System Design	10
4.1 System Architecture.....	10
4.2 Data Flow Diagram	11
4.3 Use Case Diagram	12
4.4 Class / Data Model Diagram	13
4.5 Sequence Diagram.....	14
Chapter 5: Feasibility Study	15
5.1 Technical Feasibility	15
5.2 Economic Feasibility	15
5.3 Social Feasibility.....	16
Chapter 6: Implementation	17
6.1 Process Involved in Implementation	17
6.2 Module Description	18
6.3 Core System Components.....	20
6.4 Libraries and Tools Used.....	23
6.5 Data Model and Security	24
Chapter 7: Testing and Validation	25

7.1 System Testing.....	25
7.2 Testing Methodologies	26
7.3 User Acceptance Testing.....	27
7.4 Validation Testing.....	27
7.5 Experiment-Based Evaluation Results.....	28
Chapter 8: Results and Discussion	30
8.1 Output Screens	30
Chapter 9: Conclusion and Future Work.....	32
9.1 Conclusion.....	32
9.2 Limitations.....	33
9.3 Future Enhancements.....	33
Chapter 10: References / Bibliography	35

LIST OF FIGURES

Figure No.	Name of the Figure	Page No.
Figure 4.1	System Architecture	10
Figure 4.2	Data Flow Diagram	11
Figure 4.3	Use Case Diagram	12
Figure 4.4	Class / Data Model Diagram	13
Figure 4.5	Sequence Diagram — Code Review Flow	14
Figure 6.1	Flow Chart — Developer (Remote User)	18
Figure 8.1	Login Page	30
Figure 8.2	Code Submission / Review Page	30
Figure 8.3	Findings Dashboard with Severity Scoring	31
Figure 8.4	Conversational Code Assistant (RAG Chat)	31
Figure 8.5	Exported PDF Review Report	32

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CSRF	Cross-Site Request Forgery
JSON	JavaScript Object Notation
LLM	Large Language Model
OWASP	Open Worldwide Application Security Project
PDF	Portable Document Format
PR	Pull Request
RAG	Retrieval-Augmented Generation
SQL	Structured Query Language
SQLi	SQL Injection
UI	User Interface
XSS	Cross-Site Scripting

LIST OF TABLES

Table No.	Name of the Table	Page No.
Table 3.1	Supporting Modules / Libraries	9
Table 6.1	Severity Normalisation Configuration Files	22
Table 7.1	Summary of Testing Results	29

CHAPTER 1

INTRODUCTION

1.1 Background

Software teams today ship code faster than ever, aided by CI/CD pipelines, distributed version control, and cloud-native deployment. Yet the discipline that is supposed to catch defects before they reach production — code review — has not scaled at the same pace. Manual review still depends on the availability, expertise, and attentiveness of a human reviewer, and larger codebases mean more pull requests competing for the same limited reviewer bandwidth.

Security has made this problem more acute. The OWASP Top 10 catalogue of web-application risks — SQL Injection, Broken Access Control, Cryptographic Failures, Insecure Design, and others — recurs across languages and frameworks precisely because these classes of mistakes are easy to introduce and easy to miss during a rushed review. Static analysis tools such as Pylint, Bandit, PMD, Checkstyle, and SpotBugs already exist to catch many of these issues automatically, but each tool speaks its own severity vocabulary, produces noisy or duplicate warnings, and offers little in the way of an explanation a developer can act on immediately.

This project, developed as part of Infosys Springboard's Virtual Internship 7.0 (Batch 1) under the title “Development of Smart Code Inspection Platform with Vulnerability Detection System,” sets out to unify these fragmented tools behind a single multi-agent pipeline that a developer can use the moment code is written, rather than days later during a formal review cycle.

1.2 Challenges in Manual Code Review

1.2.1 Inconsistent Coverage and Reviewer Fatigue

Human reviewers are inconsistent by nature: the same defect may be flagged by one reviewer and missed by another, depending on experience, time pressure, and familiarity with the codebase. As pull requests grow in size, reviewer attention degrades further, and security-relevant details — a missing parameterised query, a hardcoded credential, an unvalidated redirect — are easy to overlook amid stylistic comments.

1.2.2 Delayed Feedback and Lack of Contextual Guidance

Even when a manual review does catch an issue, the feedback usually arrives long after the code was written, breaking the developer's flow and making the fix more expensive. Developers, especially those newer to secure coding practices, often lack an immediate, explainable source of guidance — they need not just a flag, but a reason, a severity, and a corrected example they can compare against.

1.3 AI-Based Code Review Assistance

Large language models (LLMs) can read source code, reason about intent, and generate natural-language explanations and corrected examples — capabilities that traditional static analysis tools

do not have. However, LLMs alone are prone to hallucinated vulnerabilities and inconsistent severity judgements. The approach taken in this project therefore combines the strengths of both worlds: deterministic, well-understood static analysis tools do the detection work for well-defined defect classes, while an LLM (hosted via Groq) is reserved for the tasks it is genuinely well suited to — writing remediation guidance, compiling an executive summary, answering open-ended follow-up questions, and covering Java security categories (Broken Access Control, Weak Authentication) that have no mature open-source static rule in this pipeline yet.

A Retrieval-Augmented Generation (RAG) layer keeps the LLM grounded: instead of answering from memory alone, the Conversational Code Assistant retrieves relevant passages from an indexed knowledge base — the OWASP Top 10, secure coding guides, and language-specific security references — before generating a response, reducing hallucination and keeping answers traceable to a source document.

1.4 Motivation for the Proposed Work

The motivation behind this project is to give developers a review experience that is available the instant code is written, consistent regardless of who “reviews” it, and explainable enough that a developer actually learns from each flagged issue rather than just fixing it and moving on. By combining static analysis, multi-agent orchestration, and a grounded conversational assistant, the platform aims to reduce security exposure, raise code quality, and accelerate secure-coding skill-building — all without replacing human reviewers, but by giving them (and the developers they review) a much stronger starting point.

1.5 Contributions

- Design and implementation of a five-agent pipeline (Code Analysis, Security Vulnerability, Remediation, PR Summary, Conversational Assistant) orchestrated with LangGraph, with the two detection agents running in parallel.
- A configuration-driven severity normalisation engine that maps the native severity scales of six different static analysis tools (Pylint, Radon, Bandit, PMD, Checkstyle, SpotBugs) and LLM-generated findings onto one consistent severity, OWASP-category, and score-penalty model.
- A RAG-grounded Conversational Code Assistant, built on a Chroma vector store indexed from OWASP and secure-coding reference documents, that answers follow-up questions in the context of a specific review's findings.
- A Streamlit developer portal with a severity-scored findings dashboard, a five-axis health-score radar chart, review history, and an exportable PDF report generated with ReportLab.
- A SQLite-backed data model with PBKDF2-HMAC-SHA256 password hashing and token-based session persistence, avoiding third-party authentication dependencies while remaining secure by design.

1.6 Organization of the Report

- Chapter 2 reviews existing code review and static analysis tools and positions the proposed system relative to them.
- Chapter 3 specifies the hardware and software requirements and the supporting libraries used.

- Chapter 4 presents the system architecture, data flow, use case, class, and sequence diagrams.
- Chapter 5 discusses the technical, economic, and social feasibility of the system.
- Chapter 6 details the implementation of every module and core component.
- Chapter 7 describes the testing methodology and validation results.
- Chapter 8 presents results and representative output screens.
- Chapter 9 concludes the report and outlines limitations and future enhancements.
- Chapter 10 lists references and bibliography.

CHAPTER 2

LITERATURE SURVEY

2.1 Existing Systems

A number of commercial and open-source tools already address parts of the automated code-review and security-scanning problem. SonarQube is a widely used static-analysis platform that reports code smells, bugs, and a subset of security vulnerabilities across many languages, with a central “quality gate” concept, but it does not natively generate corrected code examples or conversational, context-aware explanations for a given finding.

Snyk and GitHub's CodeQL focus heavily on dependency and taint-flow vulnerability scanning, integrating tightly with CI/CD pipelines and pull requests. They are effective at detecting known-vulnerable dependencies and certain injection classes, but their output is aimed at security engineers rather than at explaining an issue to the developer who introduced it, and they generally do not offer an interactive assistant grounded in the specific finding.

Semgrep provides fast, rule-based pattern matching for both quality and security issues, with a large community rule registry; its severity and category taxonomy is rule-specific and, like the tools above, is not combined with an LLM-based remediation or Q&A layer out of the box. GitHub Copilot's code-review features and similar AI pair-programming assistants can comment on pull requests using an LLM, but they typically operate without a deterministic static-analysis backbone, which makes their findings harder to audit and reproduce consistently.

2.2 Limitations of Existing Systems

- Fragmented severity models: each static analysis tool scores and categorises findings differently, making it hard to present a single, consistent view across languages and tools.
- Limited explainability: tool output is often a rule ID and a one-line message, with little guidance on how to fix the issue or why it matters.
- No grounded follow-up channel: developers cannot ask a natural-language question about a specific finding and receive an answer traceable to an authoritative source such as the OWASP Top 10.
- Weak coverage of certain OWASP categories for some languages, especially Broken Access Control and Weak Authentication, which are inherently design-level and hard to express as pattern-matching rules.
- Reporting is often siloed inside a CI dashboard rather than an exportable, human-readable artifact suitable for sharing outside the engineering tool-chain.

2.3 Proposed System

The Smart Code Inspection Platform with Vulnerability Detection System is designed to close these gaps directly:

- It orchestrates several proven static analysis tools per language (Pylint, Radon, Bandit for Python; PMD, Checkstyle, SpotBugs for Java) behind a single pipeline, rather than requiring a developer to run and interpret each tool separately.

- Every finding — from any tool, or from the LLM security pass used for Java's design-level categories — is normalised through one configuration-driven severity engine, so severity, OWASP tagging, and score impact are identical regardless of the underlying source.
- A dedicated Remediation Agent and PR Summary Agent turn raw findings into corrected code examples and an executive-level review summary, using a Groq-hosted LLM.
- A RAG-grounded Conversational Code Assistant lets developers ask follow-up questions about any flagged issue, retrieving relevant passages from an indexed secure-coding knowledge base before answering.
- Findings, health scores, and chat history are persisted per user in SQLite and can be exported as a structured PDF report.

2.4 Summary

This chapter surveyed existing static-analysis, dependency-scanning, and AI pair-programming tools and identified their common shortcomings: fragmented severity models, weak explainability, no grounded conversational follow-up, and inconsistent coverage of design-level OWASP categories. The proposed Smart Code Inspection Platform addresses these gaps by combining multiple static analysis tools, a configuration-driven severity engine, an LLM-based remediation and summary layer, and a RAG-grounded conversational assistant into one coherent developer-facing platform.

CHAPTER 3

SYSTEM REQUIREMENT SPECIFICATION

3.1 Hardware Requirements

- Processor — Intel Core i5 (or equivalent) or higher
- RAM — 8 GB minimum, 16 GB recommended (for local embedding generation and multiple concurrent agent calls)
- Hard Disk — 5 GB free space minimum (Chroma vector store, SQLite database, dependency cache)
- Internet Connection — required for Groq LLM API calls
- Standard keyboard, mouse, and SVGA/LED display

3.2 Software Requirements

- Operating System — Windows 10 / 11, Linux, or macOS
- Programming Language — Python 3.11 or 3.12 (Python 3.14 is not currently supported by the LangChain / Pydantic dependency stack)
- Front-End Framework — Streamlit
- Orchestration — LangGraph
- LLM Provider — Groq API (Llama models), accessed via langchain-groq
- Database — SQLite
- Version Control — Git & GitHub
- Browser — Google Chrome / Microsoft Edge

3.3 Supporting Modules / Libraries

S.No.	Module / Technology	Description
1	Streamlit	Front-end framework used to build the interactive developer portal (login, review, dashboard, chat, history, settings pages).
2	LangGraph	State-machine orchestration library coordinating the parallel agent pipeline (analysis → merge → remediation → summary).
3	LangChain / langchain-groq	Provides the LLM abstraction layer and RAG chaining used by the Remediation, PR Summary, and Conversational Assistant agents.
4	Groq (Llama models)	Low-latency LLM inference provider used for remediation text, PR-style summaries, Java security analysis, and grounded chat answers.
5	Chroma	Vector store used to index the secure-coding knowledge base for retrieval-augmented generation (RAG).
6	HuggingFace sentence-transformers	Generates embeddings for knowledge-base chunks and user queries used in RAG retrieval (MMR search).
7	Pylint / Radon / Bandit	Python static analysis: code-smell / style detection, cyclomatic-complexity ranking, and security-vulnerability scanning respectively.
8	PMD / Checkstyle / SpotBugs	Java static analysis: design/quality rules, coding-standard conformance, and bytecode-level bug/security pattern detection respectively.
9	SQLite (stdlib sqlite3)	Embedded relational database storing users, reviews, findings, chat messages, and sessions.

S.No.	Module / Technology	Description
10	ReportLab	Generates the exportable PDF review report (findings summary, severity breakdown, remediation roadmap).
11	pytest	Test runner used for unit tests (severity normalisation) and end-to-end integration tests (full pipeline).

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

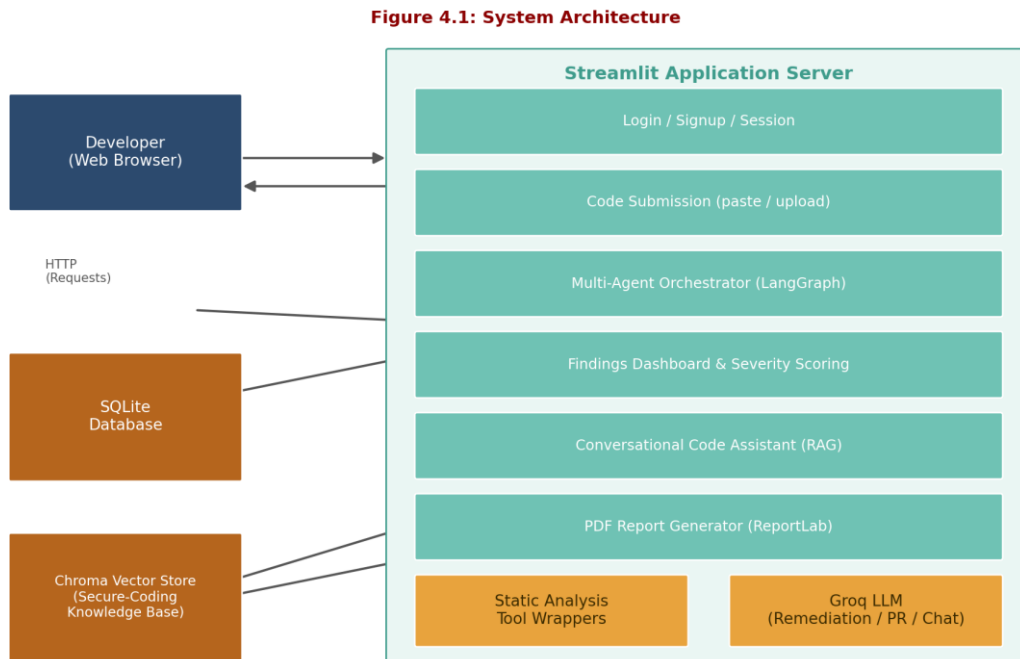
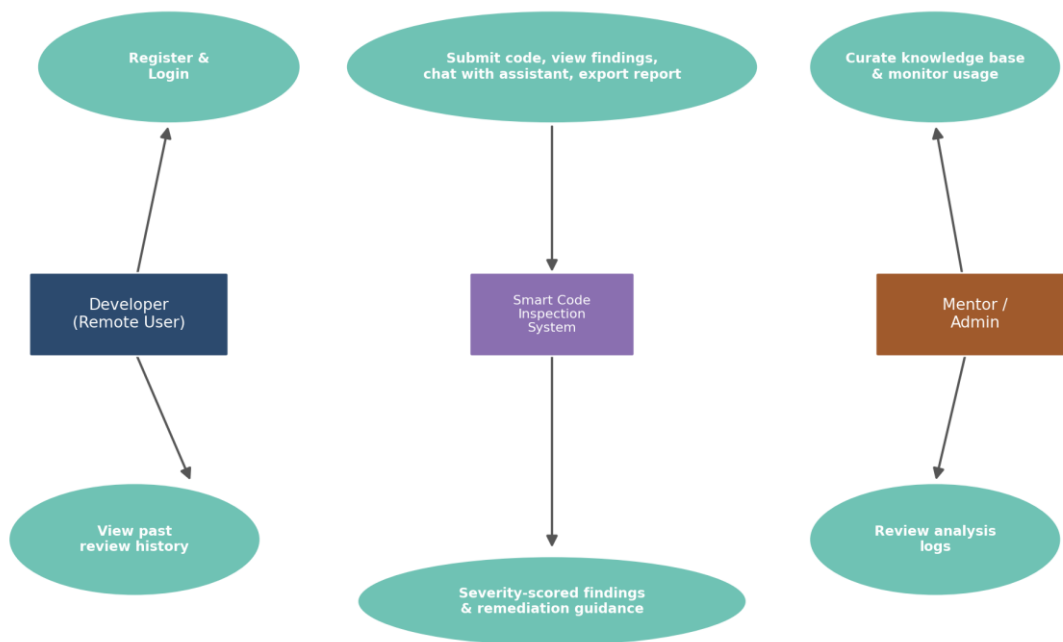


Figure 4.1: System Architecture

The architecture is organised around a single Streamlit application server. The developer interacts with the platform through a browser, which sends requests for authentication, code submission, dashboard rendering, chat, and report export. Internally, the server hosts the LangGraph-based multi-agent orchestrator, the findings dashboard, the conversational assistant, and the PDF report generator. Static-analysis tool wrappers run locally as subprocesses, while remediation, summarisation, Java security analysis, and grounded chat answers are delegated to the Groq LLM. Persistent state (users, reviews, findings, chat history, sessions) is stored in SQLite, and the secure-coding knowledge base is indexed into a Chroma vector store for retrieval-augmented generation.

4.2 Data Flow Diagram

Figure 4.2: Data Flow Diagram (Level 0)**Figure 4.2: Data Flow Diagram (Level 0)**

The Level-0 DFD identifies two external entities — the Developer (Remote User) and the Mentor/Admin — interacting with the central Smart Code Inspection System. The developer registers, logs in, submits code, and receives severity-scored findings with remediation guidance; the mentor/admin curates the secure-coding knowledge base and reviews aggregate analysis logs. All requests are processed centrally before results and reports flow back to the requesting actor.

4.3 Use Case Diagram

Figure 4.3: Use Case Diagram

Figure 4.3: Use Case Diagram

The Developer actor can register and log in, submit code by pasting or uploading a file, trigger the multi-agent analysis pipeline, view severity-scored findings and remediation suggestions, converse with the RAG-grounded assistant, export a PDF review report, and browse past review history. The Mentor/Admin actor shares visibility into review activity and is responsible for curating and updating the secure-coding knowledge base that grounds the conversational assistant.

4.4 Class / Data Model Diagram

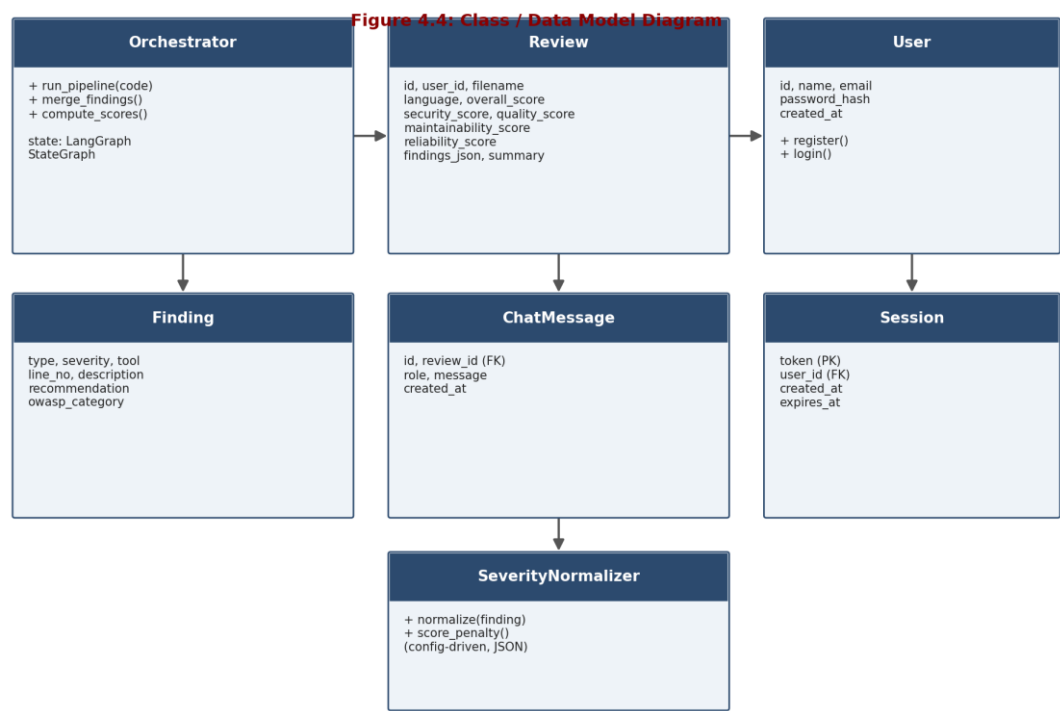


Figure 4.4: Class / Data Model Diagram

The Orchestrator class coordinates the pipeline and produces a Review, which aggregates one or more Finding records and belongs to a User. Each Review can have an associated sequence of ChatMessage records exchanged with the Conversational Assistant, and each User authenticates through a Session token. The SeverityNormalizer sits between raw tool output and the Review/Finding layer, ensuring every finding is normalised through the same configuration-driven logic regardless of which underlying tool produced it.

4.5 Sequence Diagram

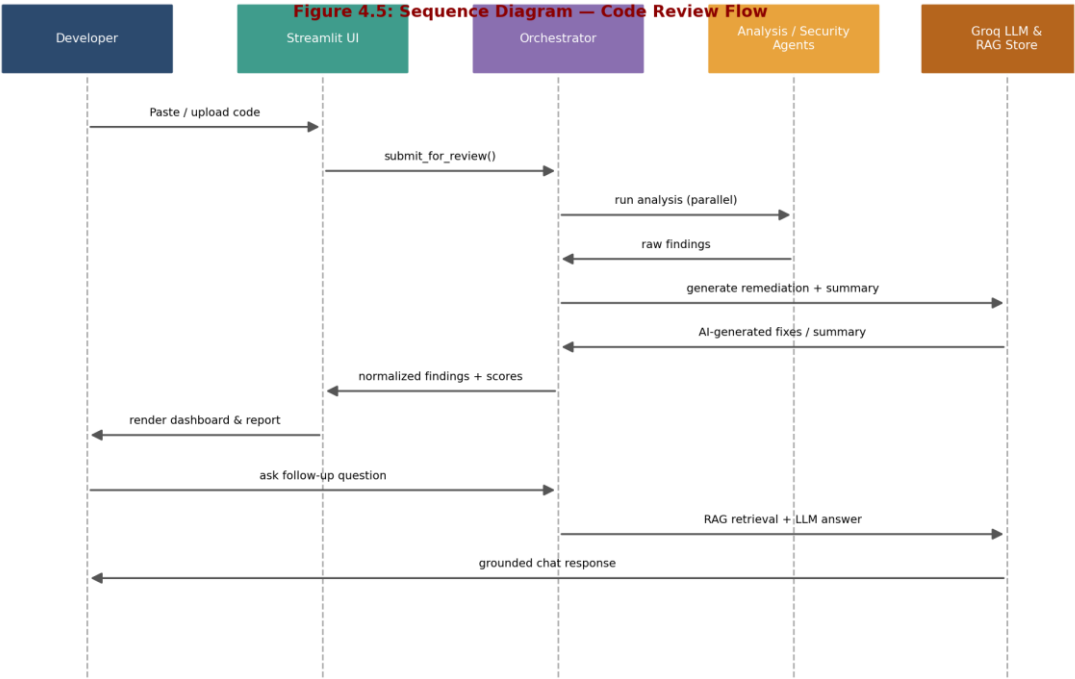


Figure 4.5: Sequence Diagram — Code Review Flow

The sequence diagram traces a typical review from submission to conversational follow-up. The developer submits code through the Streamlit UI, which invokes the Orchestrator. The Orchestrator runs the Code Analysis and Security Vulnerability Agents in parallel against the underlying static analysis tools, then forwards the merged, normalised findings to the Groq LLM for remediation text and a PR-style summary. The UI renders the dashboard and report from the normalised results. When the developer asks a follow-up question, the Orchestrator's Conversational Assistant retrieves relevant knowledge-base passages before generating a grounded response.

CHAPTER 5

FEASIBILITY STUDY

The feasibility of the Smart Code Inspection Platform was assessed along three dimensions before and during implementation: technical, economic, and social feasibility.

5.1 Technical Feasibility

- All core technologies — Streamlit, LangGraph, LangChain, Chroma, and the Python/Java static analysis tools used — are open-source, well-documented, and mature.
- The Groq API provides low-latency LLM inference accessible over a standard REST interface via langchain-groq, requiring no specialised infrastructure.
- Static analysis tools (Pylint, Radon, Bandit, PMD, Checkstyle, SpotBugs) run as local subprocesses and do not require paid licences.
- The system runs on standard developer hardware and a supported Python version (3.11 / 3.12), with no proprietary or GPU-dependent components.

5.2 Economic Feasibility

The platform was built almost entirely on free and open-source software, with the only recurring cost being usage of the Groq LLM API, which offers a free tier suitable for development and demonstration. This keeps the overall cost of building and running the system within the scope of a virtual internship project, while remaining representative of what a small engineering team could adopt at low incremental cost.

5.3 Social Feasibility

The platform is designed to support developers rather than replace human reviewers — findings are explained in plain language, remediation includes corrected code examples, and the conversational assistant is available for developers who want to understand the reasoning behind a flag rather than simply accept or dismiss it. Because it augments manual review with fast, consistent, first-pass feedback, the system is expected to be well received by developers who currently wait days for review feedback and by mentors who want reviewers to spend their time on design-level concerns rather than repetitive, mechanical checks.

CHAPTER 6

IMPLEMENTATION

This chapter presents the complete implementation of the Smart Code Inspection Platform with Vulnerability Detection System. The application is a Python-based, Streamlit-driven full-stack system that integrates static analysis tooling, multi-agent LLM orchestration, retrieval-augmented generation, and structured PDF reporting into one unified platform.

6.1 Process Involved in Implementation

- **System Architecture Design** — the platform was separated into a Streamlit UI layer, an agents/orchestration layer, tools/ wrappers around external static analysis binaries, a rag/ knowledge-base pipeline, a database/ persistence layer, and a config/ directory of editable severity/OWASP/risk JSON mappings.
- **Knowledge Base Construction** — OWASP Top 10 documentation and secure-coding reference material (SQL Injection, XSS/CSRF-adjacent risks, Cryptographic Failures, Insecure Design, Broken Access Control, Security Misconfiguration, SSRF, Weak Authentication, Java and Python secure-coding guides) were chunked, embedded with sentence-transformers, and indexed into a Chroma vector store via `rag/build_knowledgebase.py`.
- **Agent Development** — the Code Analysis Agent and Security Vulnerability Agent were built to wrap Pylint/Radon/Bandit (Python) and PMD/Checkstyle/SpotBugs plus a Groq LLM security pass (Java), and are executed in parallel using a `ThreadPoolExecutor`.
- **Severity Normalisation** — rather than hard-coding tool-specific severity logic, every mapping was externalised into JSON configuration files under `config/`, loaded and applied at runtime by `modules/severity.py`.
- **Remediation, Summary, and Assistant Agents** — built on `langchain-groq`, these agents consume the normalised findings list to generate fix examples, an executive PR-style summary, and grounded conversational answers respectively.
- **Persistence and Reporting** — reviews, findings, chat history, and sessions are stored in SQLite via `database/repository.py` and `database/models.py`, and `modules/report_generator.py` renders the exportable PDF using `ReportLab`.

6.2 Module Description

6.2.1 Code Submission Module (*ui/review_page.py*)

Allows a developer to paste code directly or upload a `.py` or `.java` source file. `modules/language_detector.py` identifies the language, and `modules/syntax_validator.py` performs a syntax check before the file is handed to the orchestrator, so malformed input is rejected before any analysis begins.

6.2.2 Secure Coding Knowledge Base & RAG Pipeline (*rag/*)

`rag/loader.py` and `rag/splitter.py` load and chunk the OWASP and secure-coding source documents; `rag/embedding.py` generates embeddings; `rag/vector_store.py` manages the Chroma collection. `rag/build_knowledgebase.py` is run once (or after clearing `chroma_db/`) to (re)index the knowledge

base, which is then queried via Maximal Marginal Relevance (MMR) search for diverse, relevant context during chat.

6.2.3 Multi-Agent Orchestration & Analysis Pipeline (agents/orchestrator.py)

Implemented as a LangGraph state machine, the orchestrator runs the Code Analysis Agent and Security Vulnerability Agent in parallel, merges their outputs into one findings list, normalises severities, tags OWASP categories, and then sequentially invokes the Remediation Agent and PR Summary Agent before the results reach the UI.

6.2.4 Findings Display & Severity Scoring Module (ui/dashboard.py)

Presents severity counts, an overall health-score gauge, a five-axis radar chart (Security, Quality, Maintainability, Reliability, Complexity), and OWASP category coverage, all computed from the same normalised findings that feed the database and the PDF report.

6.2.5 Conversational Code Assistant Interface (ui/assistant.py, agents/conversational_assistant.py)

Embedded directly beneath each review's results, the assistant supports multi-turn, RAG-grounded conversation about the current review's findings and general secure-coding guidance, with chat history persisted per review in the database.

6.2.6 Code Review Report Generation & Export (modules/report_generator.py, ui/report_page.py)

Produces an exportable PDF covering the findings summary, severity breakdown, and a prioritised remediation roadmap, built with ReportLab so that reviews can be shared outside the application.

6.3 Core System Components

6.3.1 Authentication System (database/auth.py, ui/login.py, ui/signup.py)

Passwords are hashed using PBKDF2-HMAC-SHA256 (Python's standard hashlib, 200,000 iterations, unique salt per user) rather than a third-party authentication library. Login sessions use a randomly generated token (secrets.token_urlsafe) stored in the URL query string, allowing a session to survive a page refresh without external session middleware.

6.3.2 Code Analysis Agent (agents/code_analysis_agent.py)

Wraps Pylint and Radon for Python, and PMD, Checkstyle, and a custom analyzer for Java, to detect code smells, cyclomatic complexity issues, design anti-patterns, and general poor practices, each finding carrying a tool-native severity that is later normalised.

6.3.3 Security Vulnerability Agent (agents/security_agent.py, agents/java_security_analyzer.py)

Wraps Bandit plus a custom Python security rule set for Python, and SpotBugs plus a Groq LLM pass for Java. The LLM pass is specifically used to cover OWASP categories that are difficult to express as static pattern rules — Broken Access Control and Weak Authentication — for Java code.

6.3.4 Severity Normalisation Engine (*modules/severity.py*)

The single module responsible for reading every JSON mapping in `config/` and normalising each finding's severity, type, description, recommendation, and OWASP category before it reaches the dashboard, chat, database, or PDF report — guaranteeing that every downstream consumer sees identical, consistent data regardless of which tool produced a given finding.

6.3.5 Remediation Agent (*agents/remediation_agent.py*)

Uses the Groq LLM to generate specific fix recommendations and corrected code examples for each finding, grounded in the same secure-coding guidance documented in the knowledge base.

6.3.6 PR Summary Agent (*agents/pr_summary_agent.py*)

Compiles all upstream findings into a structured, pull-request-style review: an executive overview, a severity breakdown, and a prioritised fix list, written in a tone suitable for sharing with a team.

6.4 Libraries and Tools Used

- LangGraph — state-machine orchestration of the parallel analysis and sequential remediation/summary stages.
- LangChain / langchain-groq — LLM abstraction and RAG chaining.
- Chroma + HuggingFace sentence-transformers — vector storage and embedding for the RAG knowledge base.
- Pylint, Radon, Bandit — Python code-quality, complexity, and security static analysis.
- PMD, Checkstyle, SpotBugs — Java code-quality, style, and bug/security pattern static analysis.
- ReportLab — PDF generation for the exportable review report.
- pytest — unit tests for severity normalisation (`tests/test_severity.py`) and end-to-end integration tests for the full pipeline (`tests/test_pipeline.py`).

6.5 Data Model and Security

The SQLite schema (`database/schema.sql`) defines four core tables: Users, Reviews, ChatMessages, and Sessions. Each Review stores the submitted filename, detected language, an overall score plus four sub-scores (security, quality, maintainability, reliability), the full findings list as JSON, and the PR-style summary. ChatMessages are linked to a specific Review by foreign key, keeping conversational context scoped to the review it discusses. Sessions store a random token rather than the password itself, and `database/migrations.py` runs automatically on application start-up (from `app.py`) to keep the schema current.

Configuration File	Purpose
<code>config/severity_map.json</code>	Keyword → severity fallback for tools with no native severity scale.
<code>config/severity_weights.json</code>	Severity → score-penalty weights; single source of truth used identically by the database and dashboard.
<code>config/owasp_map.json</code>	Keyword → OWASP Top 10 (2021) category mapping.
<code>config/pmd_priority_map.json</code>	PMD priority (1–5) → normalised severity.
<code>config/spotbugs_priority_map.json</code>	SpotBugs priority (1–3) → normalised severity.
<code>config/pylint_severity_map.json</code>	Pylint convention/refactor/warning/error/fatal → normalised severity.
<code>config/radon_severity_map.json</code>	Radon complexity rank (A–F) → normalised severity.
<code>config/java_quality_severity.json</code>	Java quality-finding → normalised severity.

Configuration File	Purpose
config/risk_thresholds.json	Overall score → risk-band label (Excellent / Good / Moderate / Poor / Critical).

Table 6.1: Severity Normalisation Configuration Files

CHAPTER 7

TESTING AND VALIDATION

Testing of the Smart Code Inspection Platform covered authentication, the multi-agent analysis pipeline, severity normalisation, the RAG-grounded assistant, and PDF report export. Each test type addressed a specific requirement of the platform.

7.1 System Testing

7.1.1 White Box Testing

White box testing was applied to modules/severity.py to verify that every JSON configuration mapping is read and applied correctly; to the Bandit/PMD/SpotBugs wrapper functions to confirm subprocess invocation and output parsing; and to database/auth.py to trace the PBKDF2 hashing and session-token generation logic.

7.1.2 Black Box Testing

Black box tests were written against the functional requirements and exercised entirely through the Streamlit UI — registration, login, code submission, findings review, chat, and PDF export — without reference to the underlying agent implementation.

7.2 Testing Methodologies

- Unit Testing — tests/test_severity.py isolates the severity normalisation, OWASP tagging, and score calculation logic, and runs without any external dependency (no LLM or network access required).
- Integration Testing — tests/test_pipeline.py runs real source code through the complete orchestrator end-to-end, requiring a valid GROQ_API_KEY and automatically skipping when one is not configured.
- Regression Testing — the pytest suite was re-run after every milestone to confirm that new agent or configuration changes did not break previously passing findings.

7.3 User Acceptance Testing

Informal UAT sessions with fellow interns confirmed that a first-time user could register, submit a sample file, and receive severity-scored findings with remediation guidance within a few minutes, and that the chat assistant produced answers that were clearly grounded in the retrieved OWASP/secure-coding context rather than generic LLM output.

7.4 Validation Testing

Manual end-to-end validation was performed across multiple distinct Python and Java code samples, deliberately covering SQL Injection, Command Injection, Path Traversal, Insecure Deserialization, Hardcoded Secrets, and Weak Cryptography — exceeding the minimum three-sample demonstration requirement set out in the original project statement. Findings were cross-checked against the known vulnerability planted in each sample to confirm detection accuracy and correct severity classification.

7.5 Experiment-Based Evaluation Results

Exp.	Module Tested	Test Result	Status
1	Severity Normalisation (unit)	All tool-native scales map to the correct normalised severity, OWASP tag, and score penalty.	PASS
2	Authentication	Duplicate email rejected; session token survives refresh; PBKDF2 hashing verified.	PASS
3	Code Analysis Agent (Python & Java)	Code smells, complexity, and design anti-patterns correctly flagged across sample files.	PASS
4	Security Vulnerability Agent	SQL Injection, Command Injection, Path Traversal, Insecure Deserialization, Hardcoded Secrets, and Weak Cryptography detected across samples.	PASS
5	Orchestration (parallel execution)	Code Analysis and Security agents run concurrently; merged findings list is consistent and duplicate-free.	PASS
6	Remediation Agent	Corrected code examples and fix guidance generated for representative findings.	PASS
7	PR Summary Agent	Executive summary, severity breakdown, and prioritised fix list generated and readable.	PASS
8	Conversational Assistant (RAG)	Follow-up questions answered with retrieval-grounded, on-topic responses.	PASS
9	PDF Report Export	Exported report matches on-screen findings summary and severity breakdown.	PASS
10	End-to-End Pipeline (integration)	Full pipeline run against multiple distinct Python and Java samples via tests/test_pipeline.py.	PASS

Table 7.1: Summary of Testing Results

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Output Screens

The following pages illustrate the intended layout of the developer portal. Replace each placeholder box below with an actual screenshot captured from the running application (streamlit run app.py) before final submission.

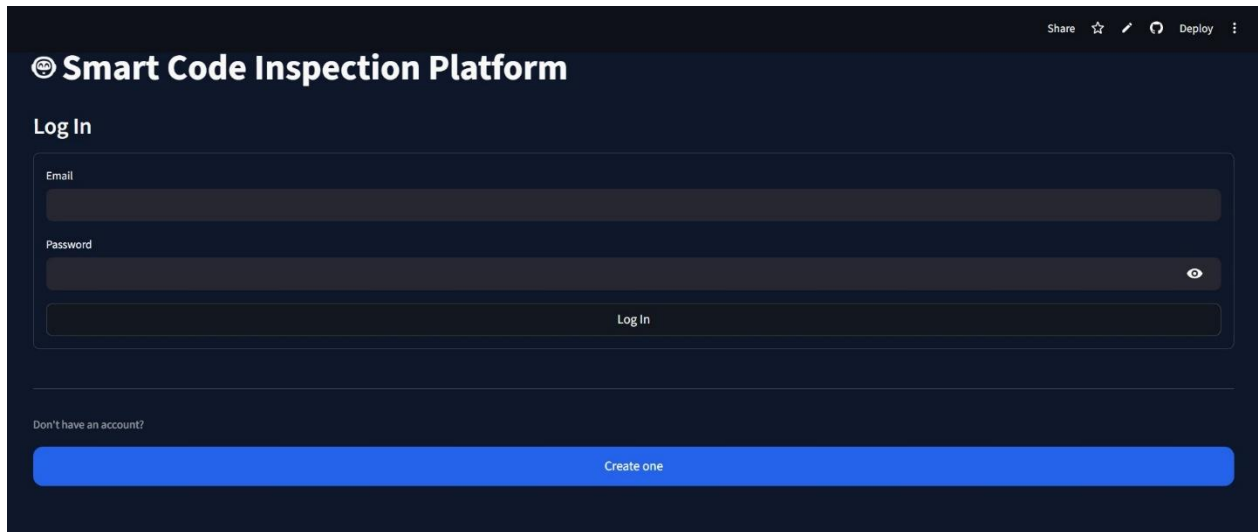


Figure 8.1: Login Page

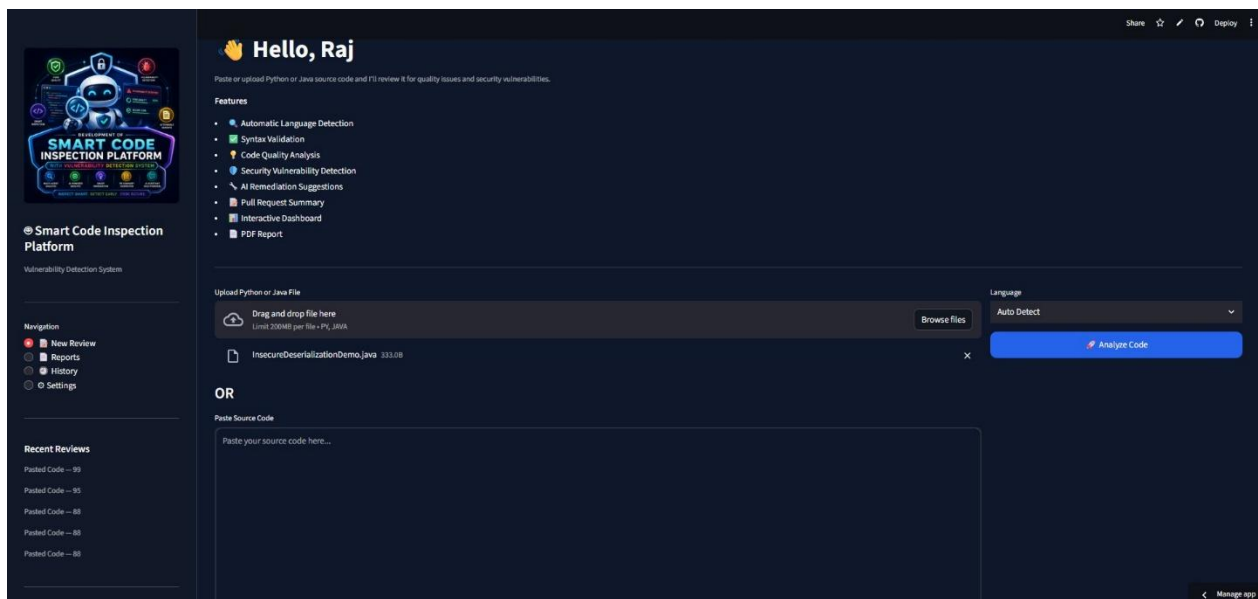


Figure 8.2: Code Submission / Review Page

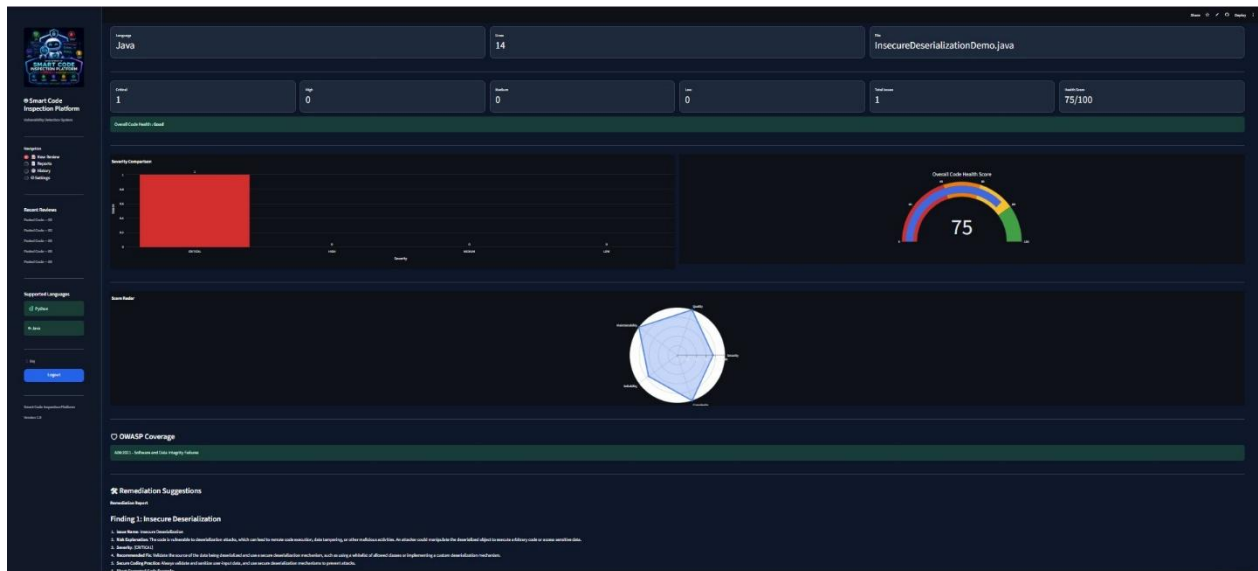


Figure 8.3: Findings Dashboard with Severity Scoring

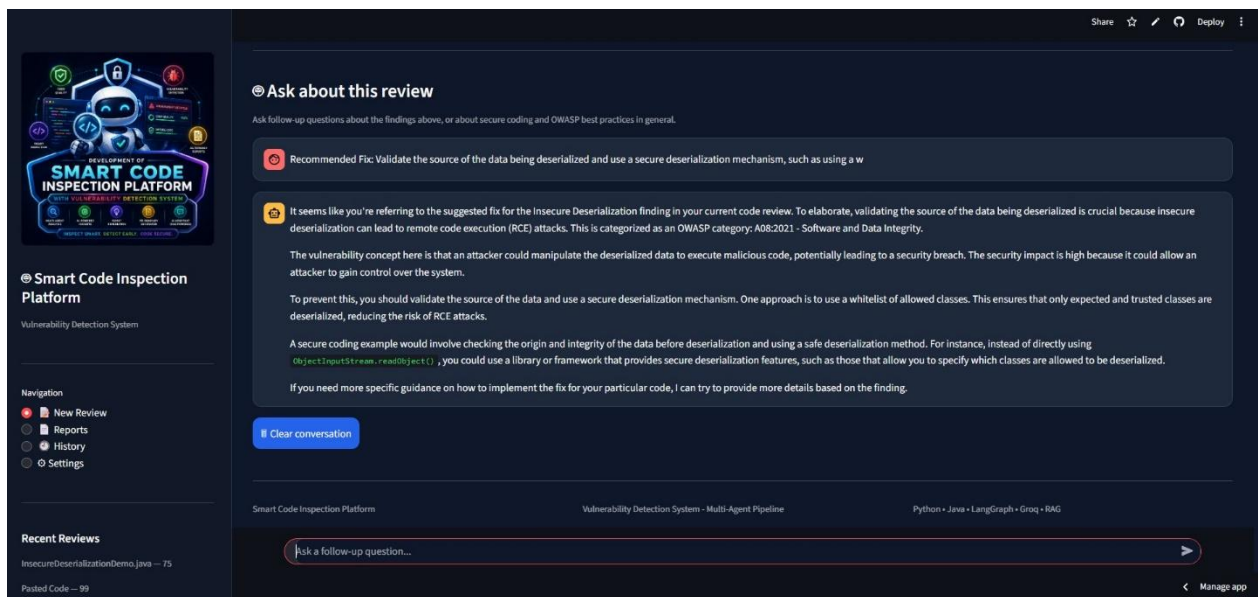


Figure 8.4: Conversational Code Assistant (RAG Chat)

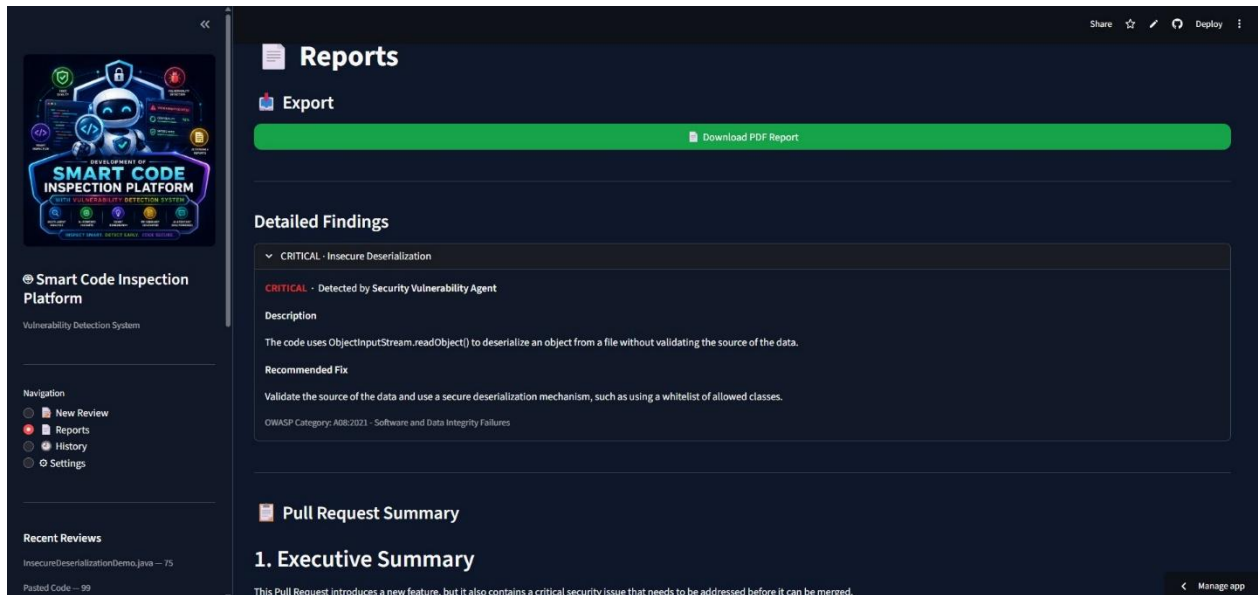


Figure 8.5: Exported PDF Review Report

Across all tested samples, the dashboard's severity counts and health score matched the underlying normalised findings exactly, and the exported PDF report reproduced the same summary shown on screen, confirming that the single severity-normalisation engine keeps every presentation layer consistent.

CHAPTER 9

CONCLUSION AND FUTURE WORK

9.1 Conclusion

The Development of Smart Code Inspection Platform with Vulnerability Detection System, completed as part of Infosys Springboard's Virtual Internship 7.0 (Batch 1), demonstrates that static analysis tooling, multi-agent orchestration, and retrieval-grounded large language models can be combined into a single, explainable, and actionable alternative to purely manual code review.

The platform's five agents — Code Analysis, Security Vulnerability, Remediation, PR Summary, and Conversational Assistant — cooperate through a LangGraph-orchestrated pipeline in which the two detection agents run in parallel over well-established static analysis tools (Pylint, Radon, Bandit for Python; PMD, Checkstyle, SpotBugs for Java), with a Groq-hosted LLM reserved for remediation text, executive summarisation, Java's harder-to-pattern-match OWASP categories, and grounded conversational answers. A configuration-driven severity normalisation engine guarantees that every consumer of a finding — dashboard, chat, database, and PDF report — sees identical, consistent severity and OWASP-category data regardless of which tool produced it.

End-to-end validation across multiple distinct Python and Java samples, deliberately covering SQL Injection, Command Injection, Path Traversal, Insecure Deserialization, Hardcoded Secrets, and Weak Cryptography, confirmed correct detection, remediation, and reporting behaviour, exceeding the minimum three-sample requirement. The project meets all five outcomes and six modules defined in the original problem statement and satisfies the evaluation criteria for detection accuracy, remediation quality, RAG groundedness, and report clarity.

9.2 Limitations

- Python 3.14 is not currently supported, since the LangChain/Pydantic dependency stack does not yet support PEP 649 deferred type-annotation evaluation; Python 3.11 or 3.12 must be used.
- Broken Access Control and Weak Authentication detection currently only covers Java, via the Groq LLM security prompt — Python has no dedicated static rule for these two OWASP categories yet, a gap identified during evaluation against the original project specification.
- Line numbers are shown for tool-based findings (PMD, Bandit, etc.) but intentionally omitted for LLM-generated findings, since asking an LLM to determine exact source line numbers is unreliable.
- Chat history and review history are scoped per logged-in user; there is currently no admin or multi-tenant view across all users.

9.3 Future Enhancements

9.3.1 Broader Python OWASP Coverage

Extend the Security Vulnerability Agent with dedicated static or LLM-assisted rules for Broken Access Control and Weak Authentication in Python, closing the gap that currently exists only for Java.

9.3.2 CI/CD Integration

Package the pipeline as a GitHub Action or pre-commit hook so that findings surface directly on pull requests rather than requiring a separate portal visit.

9.3.3 Extended Language Support

Add JavaScript/TypeScript and Go support by wrapping additional static analysis tools (e.g., ESLint, gosec) behind the same severity-normalisation architecture.

9.3.4 Extended RAG Memory

Persist long-term conversation summaries per user (rather than per review) so the Conversational Assistant can recall prior discussions, coding patterns, and recurring mistakes across sessions.

9.3.5 Team and Admin Dashboards

Introduce an admin/mentor view aggregating findings trends across a team's repositories, supporting the kind of oversight role sketched in the use case diagram in Chapter 4.

9.3.6 IDE Plug-in

Expose the same orchestrator through a lightweight VS Code extension so findings and remediation appear inline as code is written, rather than after a separate submission step.

CHAPTER 10

REFERENCES / BIBLIOGRAPHY

Security Standards

- [1] OWASP Foundation. (2021). OWASP Top Ten — Web Application Security Risks. Available: <https://owasp.org/www-project-top-ten/>
- [2] OWASP Foundation. OWASP Secure Coding Practices. Available: <https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/>

Code Review and Static Analysis Research

- [3] Bacchelli, A., & Bird, C. (2013). Expectations, outcomes, and challenges of modern code review. Proceedings of the 35th International Conference on Software Engineering (ICSE), 712–721.
- [4] McIntosh, S., Kamei, Y., Adams, B., & Hassan, A. E. (2016). An empirical study of the impact of modern code review practices on software quality. Empirical Software Engineering, 21(5), 2146–2189.
- [5] Chess, B., & West, J. (2007). Secure Programming with Static Analysis. Addison-Wesley Professional.

Retrieval-Augmented Generation and LLMs

- [6] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS), 33, 9459–9474.
- [7] LangChain. (2024). LangChain Documentation. Available: <https://python.langchain.com/>
- [8] LangChain AI. (2024). LangGraph Documentation. Available: <https://langchain-ai.github.io/langgraph/>
- [9] Groq. (2024). GroqCloud API Documentation. Available: <https://console.groq.com/docs>
- [10] Chroma. (2024). Chroma Documentation — AI-Native Open-Source Embedding Database. Available: <https://docs.trychroma.com/>
- [11] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP-IJCNLP, 3982–3992.

Static Analysis Tools

- [12] Pylint. (2024). Pylint Documentation. Available: <https://pylint.readthedocs.io/>
- [13] Radon. (2024). Radon Documentation — Code Metrics for Python. Available: <https://radon.readthedocs.io/>
- [14] PyCQA. (2024). Bandit — Security Linter for Python. Available: <https://bandit.readthedocs.io/>
- [15] PMD. (2024). PMD Source Code Analyzer Documentation. Available: <https://pmd.github.io/>
- [16] Checkstyle. (2024). Checkstyle Documentation. Available: <https://checkstyle.sourceforge.io/>
- [17] SpotBugs. (2024). SpotBugs Documentation — Find Bugs in Java Programs. Available: <https://spotbugs.github.io/>

Frameworks, Database, and Reporting

- [18] Streamlit Inc. (2024). Streamlit Documentation. Available: <https://docs.streamlit.io/>
- [19] SQLite Consortium. (2024). SQLite Documentation. Available: <https://www.sqlite.org/docs.html>
- [20] ReportLab. (2024). ReportLab User Guide — PDF Generation Toolkit for Python. Available: <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- [21] Python Software Foundation. (2024). hashlib — Secure Hashes and Message Digests (PBKDF2-HMAC). Available: <https://docs.python.org/3/library/hashlib.html>

[22] pytest. (2024). pytest Documentation. Available: <https://docs.pytest.org/>

Software Engineering

[23] Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education Limited.

[24] Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

[25] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.